

CHAPTER 6

PROJECT DEMONSTRATION

- Dataset Importation:

This code snippet imports the dataset, typically from a CSV file, into the code environment for analysis. Using the specified file path, we load the dataset into a DataFrame format, which provides a table-like structure for easy manipulation.

```
In [2]: 1 # Importing Dataset
        2 import pandas as pd
        3
        4 # Load dataset
        5 data = pd.read_csv('C:/Users/Win-10/Documents/PredictiveMaintenance/data/predictive_maintenance_dataset.csv/predictivemaint
        6
        7 # Set the display option to show all rows
        8 pd.set_option('display.max_rows', None)
        9
        10 # Display all rows of the dataset
        11 print(data)
```

Output: The dataset is successfully loaded into a DataFrame, displaying rows and columns in a structured format, ready for preprocessing and further analysis.

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	\
0	1	M14860	M	298.1	308.6	
1	2	L47181	L	298.2	308.7	
2	3	L47182	L	298.1	308.5	
3	4	L47183	L	298.2	308.6	
4	5	L47184	L	298.2	308.7	
5	6	M14865	M	298.1	308.6	
6	7	L47186	L	298.1	308.6	
7	8	L47187	L	298.1	308.6	
8	9	M14868	M	298.3	308.7	
9	10	M14869	M	298.5	309.0	
10	11	H29424	H	298.4	308.9	
11	12	H29425	H	298.6	309.1	
12	13	M14872	M	298.6	309.1	
13	14	M14873	M	298.6	309.2	
14	15	L47194	L	298.6	309.2	
15	16	L47195	L	298.6	309.2	
16	17	M14876	M	298.6	309.2	
17	18	M14877	M	298.7	309.2	
18	19	L47196	L	298.6	309.2	

- Data Preprocessing:

In this step, we clean the raw data by handling missing values, removing noise, and correcting any inconsistencies. This ensures that the dataset is reliable and ready for analysis.

```
In [3]: 1 #Data Preprocessing
2 # Check for missing values
3 missing_values = data.isnull().sum()
4 print(missing_values)
5
6 # Drop rows with missing target values (if any)
7 data = data.dropna(subset=['Machine failure'])
8
9 # Optionally, fill missing values in other columns with the mean or median
10 data.fillna(data.mean(numeric_only=True), inplace=True)
```

Output: A cleaned version of the dataset with missing values addressed, either by filling or removing them, along with other potential issues resolved, making it analysis-ready.

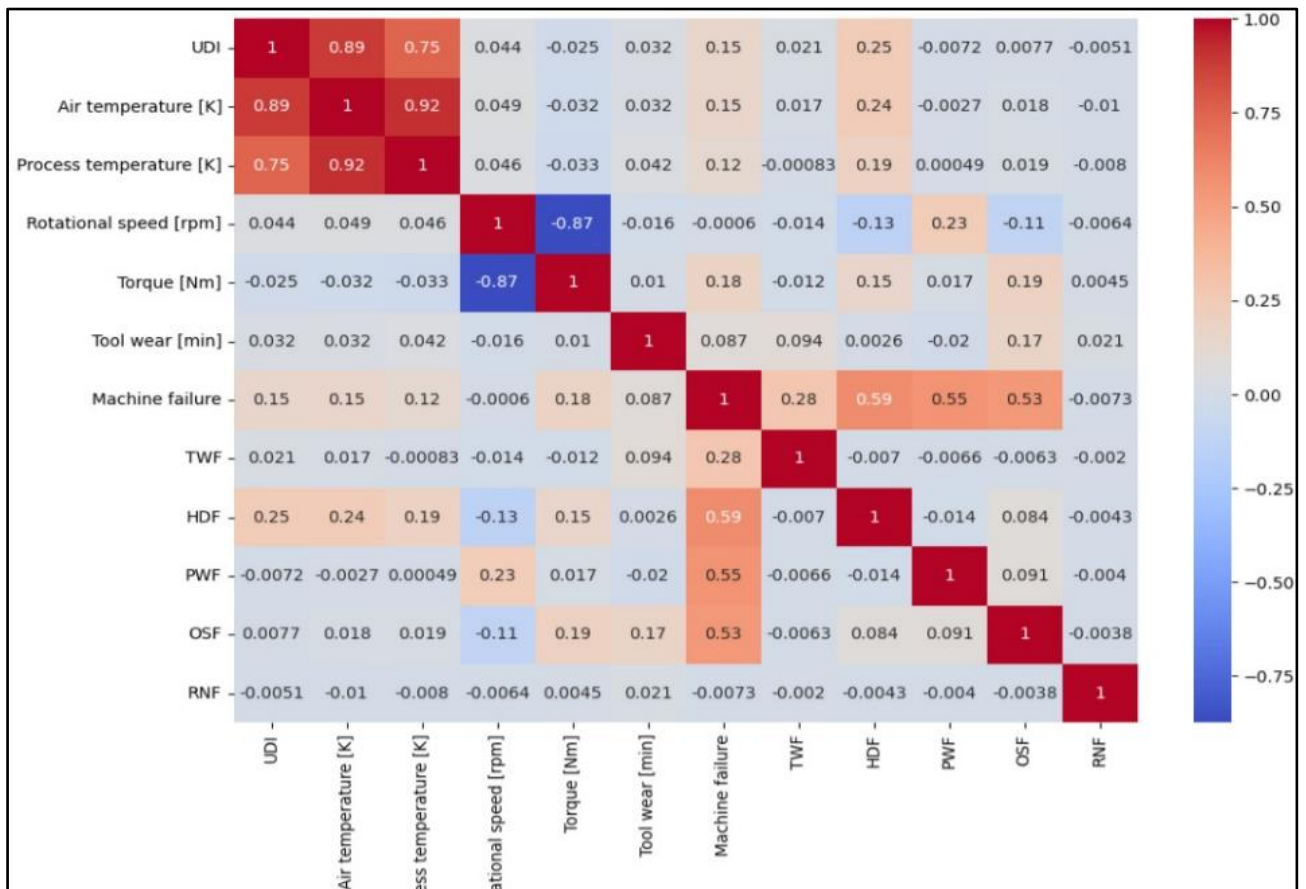
	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	\
0	1	M14860	M	298.1	308.6	
1	2	L47181	L	298.2	308.7	
2	3	L47182	L	298.1	308.5	
3	4	L47183	L	298.2	308.6	
4	5	L47184	L	298.2	308.7	
5	6	M14865	M	298.1	308.6	
6	7	L47186	L	298.1	308.6	
7	8	L47187	L	298.1	308.6	
8	9	M14868	M	298.3	308.7	
9	10	M14869	M	298.5	309.0	
10	11	H29424	H	298.4	308.9	
11	12	H29425	H	298.6	309.1	
12	13	M14872	M	298.6	309.1	
13	14	M14873	M	298.6	309.2	
14	15	L47194	L	298.6	309.2	
15	16	L47195	L	298.6	309.2	
16	17	M14876	M	298.6	309.2	
17	18	M14877	M	298.7	309.2	
18	19	L47196	L	298.6	309.2	

- Feature Selection:

Feature selection identifies the most relevant columns for predicting the target variable, helping streamline the model's inputs. By removing irrelevant features, we improve the model's efficiency and accuracy.

```
In [5]: 1 #Feature Selection
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4
5 # Define the list of numerical features with the exact column names
6 numerical_features = [
7     'UDI', 'Air temperature [K]', 'Process temperature [K]', 'Rotational speed [rpm]', 'Torque [Nm]',
8     'Tool wear [min]', 'Machine failure', 'TWF', 'HDF', 'PWF', 'OSF', 'RNF']
9
10 # Compute the correlation matrix for numerical features only
11 correlation_matrix = data[numerical_features].corr()
12
13 # Plot the correlation matrix
14 plt.figure(figsize=(12, 8))
15 sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
16 plt.show()
```

Output: A refined dataset with only the most relevant features retained, ready to be split into training and testing datasets.



```
In [6]: 1 #Feature Selection
2 # Based on the correlation analysis, select features
3 # Assuming that we select features that have a high correlation with 'Machine failure'
4 selected_features = [
5     'Air temperature [K]', 'Process temperature [K]',
6     'Rotational speed [rpm]', 'Torque [Nm]', 'Tool wear [min]'
7 ]
8
9 # Prepare the feature set X and the target variable y
10 X = data[selected_features]
11 y = data['Machine failure']
```

- Data Splitting:

It divides the data into two sets: one for training your model and one for testing it.

It takes the full dataset as input.

And then gives two separate datasets: one for training (fitting the model) and one for testing (evaluating the model) as an output

```
In [8]: 1 #Data Splitting
2 from sklearn.model_selection import train_test_split
3
4 # Split the data into training and testing sets (80% training, 20% testing)
5 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
6
7 print(f'Training set size: {len(X_train)}')
8 print(f'Testing set size: {len(X_test)}')
```

Training set size: 1924
Testing set size: 482

Two distinct datasets: one for training the model and another for testing its accuracy and performance.

- Feature Scaling:

Feature scaling standardizes the values of each feature, ensuring they all contribute equally to the model. By normalizing the range, we prevent any single feature from disproportionately affecting the model.

```
In [10]: 1 #Feature Scaling
2 from sklearn.preprocessing import StandardScaler
3
4 # Assuming X_train and X_test were prepared from the selected features in the previous step
5 # Initialize the scaler
6 scaler = StandardScaler()
7
8 # Fit the scaler on the training data and transform both training and testing data
9 X_train_scaled = scaler.fit_transform(X_train)
10 X_test_scaled = scaler.transform(X_test)
11
12 # Display the first few rows of the scaled data to check
13 print("Scaled Training Data (first 5 rows):")
14 print(X_train_scaled[:5])
15 |
16 print("Scaled Testing Data (first 5 rows):")
17 print(X_test_scaled[:5])
```

Output: Scaled training and testing datasets where feature values are transformed to similar ranges, allowing for balanced model learning.

Scaled Training Data (first 5 rows):

```
[[ 0.00482741  0.79857836 -0.49500287  0.01743195 -0.37037073]
 [ 0.04782714  0.1341481  -0.92537711  0.87199487  0.15838621]
 [ 1.98281479  1.40260587  0.13149129  0.63070652 -1.07019608]
 [-0.03817231  0.61737011 -0.1354497   0.13807613 -0.71250757]
 [-0.12417176 -0.7114904   0.40932782 -0.82707728  0.98262496]]
```

Scaled Testing Data (first 5 rows):

```
[[ 2.15481369  2.00663337 -0.16813635 -0.09315854  0.11173118]
 [-0.21017122  0.0133426   4.94187683 -2.61663255 -1.00798939]
 [ 0.04782714  0.73817561 -0.44052511  0.6910286  -0.07488891]
 [ 2.19781341  2.18784162  1.46619622 -1.50067393  1.13814171]
 [-0.8551671  -0.7114904  -0.90903379  1.16355163 -0.02823389]]
```

- Model Training:

In this step, the training dataset is used to train the predictive model. By analyzing patterns within the data, the model "learns" how to make predictions based on the input features.

```
In [8]: 1 #Model Training
        2 from sklearn.ensemble import RandomForestClassifier
        3
        4 # Initialize the model
        5 model = RandomForestClassifier(random_state=42)
        6
        7 # Train the model
        8 model.fit(X_train_scaled, y_train)
```

Output: A trained model that is capable of making predictions based on learned data patterns.

```
Out[8]: ▼ RandomForestClassifier
        RandomForestClassifier(random_state=42)
```

- Model Evaluation:-

Model evaluation assesses how well the trained model performs on the test dataset. Using metrics like accuracy, confusion matrix, and classification report, we evaluate the model's predictive quality and reliability.

```
In [9]: 1 #Model Evaluation
2 from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
3
4 # Make predictions on the test set
5 y_pred = model.predict(X_test_scaled)
6
7 # Evaluate the model
8 print(f'Accuracy: {accuracy_score(y_test, y_pred)}')
9 print('Confusion Matrix:')
10 print(confusion_matrix(y_test, y_pred))
11 print('Classification Report:')
12 print(classification_report(y_test, y_pred))
```

Accuracy: 0.9875518672199171

Confusion Matrix:

```
[[461  0]
 [ 6 15]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.99	1.00	0.99	461
1	1.00	0.71	0.83	21
accuracy			0.99	482
macro avg	0.99	0.86	0.91	482
weighted avg	0.99	0.99	0.99	482